Deep Learning and Groups

Homework 4 — Invariant and Equivariant Neural Networks

Course 00460215
Technion ECE

Released: Tuesday, 30 June 2026 • Due: Wednesday, 22 July 2026

Submission and group policy. You may work in **groups of up to three students**. Each group submits **one** joint solution on Moodle by 23:59 on the due date and receives a single shared grade.

- **Front page.** List every group member's full name and ID number on the first page of your report. Groups of one or two are also allowed; the maximum group size is three.
- **One submission per group.** Only one member uploads on behalf of the group. Do not submit individually if you are in a group.
- **Shared responsibility.** The whole group is responsible for the entire submission. Every member must understand, and be able to explain, every solution the group submits.
- **No collaboration across groups.** You may discuss the course material with anyone, but you may not share written solutions or code with, or copy them from, another group or any external source. Cite any reference you use beyond the course notes.

What to hand in. Submit **exactly two files**:

(1) A single **PDF report** containing your written solutions to Questions 1 and 2, and all code descriptions, plots, tables, and discussion for Questions 3 and 4.

(2) A single `.zip` **archive** containing all runnable code for Questions 3 and 4: every `.py` file and/or `.ipynb` notebook, including the requested test and training functions. The code must run as submitted; state any external dependency at the top.

Name the archive `<id1>_<id2>_<id3>_hw4.zip`.

Grading. Question 1: **20 pts** · Question 2: **20 pts** · Question 3: **25 pts** · Question 4: **35 pts** (total **100**). Parts marked **Bonus** are graded on top of the 100 points, up to the cap stated next to each bonus.

Tools and reproducibility. For Questions 3 and 4 you may use PyTorch and NumPy, and any free GPU service such as Google Colab (`https://colab.research.google.com/`). Fix and report a random seed so that your numbers are reproducible.

Notation. Throughout, G is a finite group with identity e, and S_n is the symmetric group on $\{1, \dots, n\}$. A permutation $\pi \in S_n$ acts on $\mathbb{R}^n$ by coordinate permutation, $(\pi \cdot x)_i := x_{\pi^{-1}(i)}$. For a permutation g acting on a set X we write $\mathrm{Fix}_X(g) := \{\, i \in X : g(i) = i \,\}$ for its set of fixed points, and $\langle \chi, \psi \rangle := \frac{1}{|G|} \sum_{g \in G} \chi(g)\, \overline{\psi(g)}$ for the inner product of class functions.

Question 1 (Character inner products and Burnside's lemma (20 pts)). *Let $G \leq S_n$ be any subgroup, acting on $X = \{1, 2, \ldots, n\}$ in the natural way and on $\mathbb{R}^n$ by the permutation representation σ, i.e. $(\sigma(g)x)_i = x_{g^{-1}(i)}$. A linear map $W : \mathbb{R}^n \to \mathbb{R}^n$ (equivalently, an $n \times n$ matrix) is called* ***G-equivariant*** *if*

$$W\,\sigma(g)\,x = \sigma(g)\,W\,x \qquad \text{for all } g \in G,\ x \in \mathbb{R}^n.$$

Write $\mathrm{Hom}_G(\mathbb{R}^n, \mathbb{R}^n)$ for the vector space of all G-equivariant linear maps $\mathbb{R}^n \to \mathbb{R}^n$. Its dimension is exactly the number of free parameters of a G-equivariant linear layer $\mathbb{R}^n \to \mathbb{R}^n$. In Tutorial 12 we computed this number in two seemingly unrelated ways.

Method A (orbits / Burnside). *Let G act on $X^2 = X \times X$ diagonally, $g \cdot (i, j) = (g(i), g(j))$. The entries W_{ij} of a G-equivariant matrix are constant on the orbits of this action, so*

$$\dim \mathrm{Hom}_G(\mathbb{R}^n, \mathbb{R}^n) \;=\; |X^2/G| \quad (\textit{the number of orbits}).$$

Method B (irreducibles / Schur). *Decomposing σ into irreducibles and applying Schur's lemma gives*

$$\dim \mathrm{Hom}_G(\mathbb{R}^n, \mathbb{R}^n) \;=\; \langle \chi_\sigma,\, \chi_\sigma \rangle,$$

where χ_σ is the character of the permutation representation.

Problem. *Prove that the two methods always agree: for every subgroup $G \leq S_n$,*

$$|X^2/G| \;=\; \frac{1}{|G|} \sum_{g \in G} \big|\mathrm{Fix}_X(g)\big|^2 \;=\; \langle \chi_\sigma,\, \chi_\sigma \rangle.$$

Conclude that Methods A and B count the same number of G-equivariant layers $\mathbb{R}^n \to \mathbb{R}^n$, for every finite subgroup $G \leq S_n$ and every $n \in \mathbb{N}$.

Hints.

(1) For the first equality, apply Burnside's lemma (the orbit-counting theorem) to the diagonal action of G on X^2, and show that the number of pairs $(i, j) \in X^2$ fixed by g equals $|\,\mathrm{Fix}_X(g)|^2$.

(2) For the second equality, recall that the permutation character satisfies $\chi_\sigma(g) = |\,\mathrm{Fix}_X(g)|$, that χ_σ is real-valued, and that $\langle \chi_\sigma, \chi_\sigma \rangle = \frac{1}{|G|} \sum_g \chi_\sigma(g)^2$.

Question 2 (Netflix rating matrices and group equivariance (20 pts)). *A streaming service stores its data as a rating matrix $A \in \mathbb{R}^{m\times n}$, where A_{ij} is the rating that user $i \in \{1,\dots,m\}$ gives to movie $j \in \{1,\dots,n\}$. The labels of the users and of the movies carry no intrinsic order: relabeling the users (permuting the rows) and relabeling the movies (permuting the columns) describe the same underlying data.*

Throughout this question, let $G = S_m \times S_n$ act on $\mathbb{R}^{m\times n}$ by

$$\big((\pi,\tau)\cdot A\big)_{ij} \;=\; A_{\pi^{-1}(i),\,\tau^{-1}(j)}, \qquad \pi \in S_m,\ \tau \in S_n,$$

*so that π permutes the rows and τ permutes the columns independently. A linear map $L : \mathbb{R}^{m\times n} \to \mathbb{R}^{m\times n}$ is G-**equivariant** if $L\big((\pi,\tau)\cdot A\big) = (\pi,\tau)\cdot L(A)$ for all $(\pi,\tau) \in G$ and all A. A linear map $\ell : \mathbb{R}^{m\times n} \to \mathbb{R}$ is G-**invariant** if $\ell\big((\pi,\tau)\cdot A\big) = \ell(A)$ for all (π,τ) and all A.*

Answer the following. Justify every answer with a proof or a clear derivation; for the parameter counts you may use the orbit-counting / character argument from Question 1 and Tutorial 12.

(a) ***Symmetry group.*** *Explain why $G = S_m \times S_n$, acting as above, is the natural symmetry group of the data, and describe its action on a rating matrix in words.*

(b) ***Equivariant linear layer.*** *Determine the general form of a G-equivariant affine map $L(A) = L_0(A) + B$, where L_0 is linear and $B \in \mathbb{R}^{m\times n}$ is a bias. Write $L(A)_{ij}$ explicitly as a formula involving the entry A_{ij}, the row sum $\sum_l A_{il}$, the column sum $\sum_k A_{kj}$, and the total sum $\sum_{k,l} A_{kl}$. State how many free parameters the linear part L_0 has and how many the bias B has.*

(c) ***Parameter-sharing diagram.*** *View the linear part L_0 as an $(mn)\times(mn)$ matrix acting on the vectorization of A. Draw its parameter-sharing pattern: indicate which entries of this matrix are tied to the same free parameter. Equivalently, fix a single output entry (i,j) and shade, on a copy of the $m \times n$ input grid, which input entries (k,l) are multiplied by each parameter from part (b). The case $m = n = 3$ suffices to show the pattern.*

(d) ***Invariant linear layer.*** *Determine the general form of a G-invariant linear map $\ell : \mathbb{R}^{m\times n} \to \mathbb{R}$, and state how many free parameters it has.*

(e) ***Three-way tensors.*** *Now suppose the data is a 3-tensor $A \in \mathbb{R}^{m\times n\times k}$ (for example, users $\times$ movies $\times$ time), with symmetry group $S_m \times S_n \times S_k$ acting by permuting the three index sets independently. How many free parameters does a general equivariant linear map $\mathbb{R}^{m\times n\times k} \to \mathbb{R}^{m\times n\times k}$ have? Describe (and sketch) the resulting parameter-sharing scheme.* ***Hint:*** *count the orbits of the action on pairs of index triples; the three coordinates contribute independently, so compare your count with the $\mathbb{R}^n \to \mathbb{R}^n$ and $\mathbb{R}^{m\times n} \to \mathbb{R}^{m\times n}$ cases.*

Question 3 (Implementing set networks in PyTorch (25 pts)). *Consider set data with S_n symmetry. A set of n elements, each with d feature dimensions, is represented as a matrix $X \in \mathbb{R}^{n\times d}$ whose i-th row $X_i \in \mathbb{R}^d$ is the feature vector of the i-th element. A permutation $\pi \in S_n$ acts by permuting the rows, $(\pi \cdot X)_i = X_{\pi^{-1}(i)}$. Recall the two target behaviours:*

- *a network $F : \mathbb{R}^{n\times d} \to \mathbb{R}^p$ is **invariant** if $F(\pi \cdot X) = F(X)$ for all $\pi \in S_n$;*
- *a layer $f : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d'}$ is **equivariant** if $f(\pi \cdot X) = \pi \cdot f(X)$ for all $\pi \in S_n$.*

*You will implement the following architectures in PyTorch. For **each** one, supply (i) a `torch.nn.Module` class implementing it, and (ii) a **test function**. Your implementations must accept inputs of shape (n, d) with $d \geq 1$ arbitrary.*

Specification of the test functions. *First decide, for each architecture, whether it is invariant or equivariant; this determines which property the test checks. For parts (a)–(d) the test does **not** train the network: with randomly initialized parameters it must*

1. *generate a random input $X \in \mathbb{R}^{n\times d}$;*
2. *draw a random permutation $\pi \in S_n$ and form $\pi \cdot X$;*
3. *compare the network's outputs on X and on $\pi \cdot X$ (for an invariant net: $F(\pi \cdot X) \approx F(X)$; for an equivariant layer: $f(\pi \cdot X) \approx \pi \cdot f(X)$);*
4. *return a boolean: `True` iff the relevant equality holds up to a numerical tolerance (e.g. `atol=1e-5`) that you state.*

Architectures. *Throughout, "your favourite model" means any standard network you choose, e.g. an MLP applied to the flattened input; state which one you used.*

(a) ***Canonization-based network.*** *Apply a **canonization** function $c : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$ that reorders the rows by a fixed, deterministic rule (for instance, sort the rows in lexicographic order, with a deterministic tie-break), so that $c(\pi \cdot X) = c(X)$ for every π; then apply your favourite model to $c(X)$. State your canonization rule and why it is permutation-invariant.*

(b) ***Symmetrization network.*** *Symmetrize your favourite model h by averaging over the **whole** group: $F(X) = \frac{1}{n!}\sum_{\pi\in S_n} h(\pi \cdot X)$. (Because this enumerates all $n!$ permutations, run it only for small n.)*

(c) ***Sampled symmetrization.*** *Same as (b), but average over a random subset $P \subseteq S_n$ of B permutations: $F(X) = \frac{1}{|P|}\sum_{\pi\in P} h(\pi \cdot X)$. State B, and note that this network is only **approximately** invariant; choose the tolerance in your test accordingly and report it.*

(d) ***Linear equivariant layers.*** *Build a network by stacking S_n-equivariant linear layers (DeepSets layers) with pointwise nonlinearities. Recall the equivariant linear layer $f(X)_i = X_i W_1 + \big(\frac{1}{n}\sum_j X_j\big)W_2 + b$, with learnable $W_1, W_2 \in \mathbb{R}^{d\times d'}$ and $b \in \mathbb{R}^{d'}$. Test the equivariance of a single layer; if you also build an invariant network, pool over the set (e.g. sum or mean over the n rows) at the end and test invariance of the whole network.*

(e) ***Data augmentation.*** *Implement a standard (not architecturally invariant) model on the flattened input, and train it with data augmentation (apply a fresh random permutation to each example each time it is seen). The **regression target** is the coordinate-wise variance of the set,*

$$g(X) \in \mathbb{R}^d, \qquad g(X)_c \;=\; \frac{1}{n}\sum_{i=1}^{n}\big(X_{ic} - \bar{X}_c\big)^2, \quad \bar{X}_c = \frac{1}{n}\sum_{i=1}^{n} X_{ic},$$

which is permutation-invariant. Devise a full training procedure (loss, optimizer, data generation), train the model, and then report (1) the approximation error to g on held-out random

sets and (2) an ***invariance error****, e.g.* $\mathbb{E}_{X,\pi} \|F(\pi \cdot X) - F(X)\|$*, measured before and after training. Discuss whether augmentation makes the network approximately invariant.*

In your PDF report, include for each part a short description, the outcome of the test function (pass/fail and tolerance), and for (e) the two error curves or numbers.

Question 4 (Benchmarking permutation-invariant networks on 3D shape classification (35 pts))**.** *In this question you compare the approaches of Question 3 on a real task: classifying 3D point clouds, treated as sets of points, into object categories. You will evaluate accuracy and computational cost, and analyze the trade-offs.*

Learning task and data. *Classify shapes from the ModelNet40 dataset (40 object categories; 9,843 training and 2,468 testing CAD meshes, sampled to point clouds).*

- ***Download.*** *Use the ModelNet40 "normal resampled" release:*

  `https://www.dropbox.com/scl/fi/qcddtiathbfcp6v3wimoi/modelnet40_normal_resampled.zip?rlkey=3t9jz6lr954wxmfj4xp6u9m2a&dl=0`

  If this link is unavailable, any standard mirror of ModelNet40 "normal resampled" is acceptable; ***state your source*** *in the report.*

- ***Format.*** *Each point cloud is a text file under a per-class folder; each row holds six numbers: the first three are the* (x, y, z) *coordinates and the next three are the normal direction.* ***Unless stated otherwise, use the first 256 points of each cloud****, i.e. represent each cloud as a* 256×3 *matrix of coordinates. Example loader:*

  ```
  import numpy as np
  data = np.loadtxt('airplane/airplane_0001.txt', delimiter=',')[:256]
  coords = data[:, :3]  # x, y, z
  ```

- ***Sanity check.*** *Visualize several classes with a 3D scatter plot and confirm the data looks reasonable; include one or two such plots.*

Architectures. *Use permutation-invariant classifiers built from the Question 3 components:*

(a) Canonization-based network.

(b) Symmetrization network. (Full symmetrization is infeasible for 256 points; use a small number of points — possibly fewer than 10 — for which the experiments below run in reasonable time. State the number you used.)

(c) Sampled-symmetrization network. (Sample 5% *of the possible permutations; you may use more points than in (b), up to all 256.)*

(d) Linear equivariant layers with pointwise nonlinearities, followed by an invariant pooling and a classifier head.

(e) Data augmentation with a standard (non-invariant) architecture.

For the canonization-based and symmetrization networks, implement the base model as a standard MLP. ***Bonus (up to 5 pts):*** *also implement them with a Transformer plus positional encoding.*

Experimental protocol.

- *Train with* `CrossEntropyLoss`*. Fix and report a random seed.*
- ***Validation / early stopping.*** *Hold out a fixed validation set — carve a disjoint* 20% *of your selected training samples (stratified across classes) — and use it for early stopping.* ***Report final accuracy on the official ModelNet40 test set*** *(not on validation).*
- ***Training-set size.*** *Repeat training with* 5*,* 10*, and* 50 *samples per class. If these give no appreciable difference, adjust the values and say so.*
- ***Time budget.*** *Cap each single experiment at* 0.5 *hours of wall time. If an experiment exceeds the cap, mark it "Out of Time" (OOT) and report the last test accuracy reached.*

- ***Bonus (up to 3 pts):*** *repeat using the surface normals as additional input features and report the effect.*

Analysis and reporting.

1. *One plot of test accuracy vs. number of training examples per class, one curve per architecture.*
2. *A table of the (wall-clock) runtime of each architecture.*
3. *A discussion of the trade-offs between the approaches in terms of: accuracy; computational efficiency; scalability with set size and feature dimension; and ease of implementation.*
4. *The challenges you met during implementation and experimentation.*
5. *A recommendation: which approach(es) suit which scenario (e.g. small vs. large sets, scarce vs. abundant compute).*
6. *Why linear equivariant layers may be especially well suited to this task, compared with the other architectures.*
7. ***Bonus (up to 5 pts).*** *This task has a further symmetry: rotating the whole point cloud does not change its class. Consider the matrix of pairwise distances between points. Which group leaves it invariant? Use it to design rotation-invariant per-point features.* ***Suggestion:*** *first "centralize" the cloud so the points have mean zero and the point of maximal norm lies on the unit sphere.*